Project Demo Document

555 Timer Frequency Measurement with Artix-7 FPGA

Board	Invent Logic Edge A7 (Artix-7)
Timer IC	Texas Instruments SA555P
Interface	UART @ 115200 baud
Clock	50 MHz system clock
PC Decoder	Python (pyserial)

1. Project Overview

This project demonstrates how an external analog timer IC (SA555P) can be interfaced with an Artix-7 FPGA to perform digital frequency measurement. The 555 timer operates in astable mode, generating a continuous square wave. The FPGA measures the period by counting 50 MHz clock ticks between rising edges, then transmits the result over UART. A Python script on the PC decodes the hex stream and prints the live frequency in Hz.

Key concepts covered:

- Asynchronous signal synchronisation — the 2-FF synchroniser pattern
- Edge detection in hardware
- Period and frequency measurement using counters
- UART serial transmission from an FPGA
- Safe mixed-voltage interfacing (5 V to 3.3 V)
- Real-time UART decoding with Python

1.1 Learning Objectives

1. Explain why a 2-FF synchroniser is mandatory for any external asynchronous signal
2. Design an astable 555 oscillator and calculate RA, RB, C for a target frequency
3. Implement a period counter in Verilog and derive frequency from it
4. Implement a UART transmitter FSM and send multi-character data frames
5. Use PulseView to capture and verify signals from real hardware
6. Write a Python script to decode UART hex data and compute frequency in real time

2. Hardware Setup

2.1 SA555P Astable Circuit

The SA555P is powered at 5 V (within its specified 4.5–16 V range). The output swings from 0 V to approximately 4.8 V, so a resistor voltage divider brings it down to a safe level for the LVCMOS33 I/O standard of the Artix-7.

Component Values

Component	Value	Purpose
Vcc	5 V	SA555P supply (min 4.5 V per datasheet)
RA	4.7 kΩ	Timing resistor (charge path)
RB	4.7 kΩ	Timing resistor (charge + discharge path)
C	100 nF	Timing capacitor
CV bypass	10 nF	Pin 5 to GND — noise rejection on control voltage
R_div1 (series)	2 kΩ	Top half of output voltage divider
R_div2 (shunt)	3.3 kΩ	Bottom half of voltage divider to GND

Frequency Calculation

```
f = 1.44 / ((RA + 2 x RB) x C)
f = 1.44 / ((4700 + 9400) x 0.0000001) = ~1021 Hz (nominal)
```

Note: Component tolerances will shift the actual frequency. The FPGA + Python measurement gives the true real-time value.

Voltage Divider — Output Level Calculation

```
V_fpga = 5.0 x (3.3 / (2.0 + 3.3)) = 3.11 V  (safe for LVCMOS33)
```

2.2 Pin Connections

Signal	Package Pin	Notes
clk (50 MHz)	**N11**	On-board crystal oscillator
sig_555 (555 output)	**N14**	Via voltage divider. Expansion connector pin 15
rst_btn (reset)	**M14**	Centre push button (active-high, PULLDOWN)
uart_txd (UART TX)	**C4**	USB-UART TX to PC (115200 8N1)

3. Verilog Firmware

3.1 Design Architecture

- uart_tx — FSM-based UART transmitter (8N1, CLKS_PER_BIT = 434 for 50 MHz / 115200)
- top_555_freq — Top-level: 2-FF synchroniser, period counter, UART print FSM

3.2 2-FF Synchroniser

The 555 output is asynchronous to the 50 MHz FPGA clock. Two flip-flops in series resolve metastability:

```
always @(posedge clk) begin
    s0 <= sig_555;   // First FF: may go metastable
    s1 <= s0;        // Second FF: stable output
end
wire rising_edge = s0 & ~s1;
```

3.3 Period Counter

On each rising edge, period_latch captures the tick count and valid pulses for one cycle. At 50 MHz, 1 kHz gives period_latch = 50,000. At 2.6 kHz, period_latch ≈ 19,230.

3.4 UART Print FSM

When valid fires, sends period_latch as 8 hex ASCII characters + CR + LF. ASCII conversion: nibble < 10 adds 0x30, nibble >= 10 adds 0x37.

4. Demo Procedure

4.1 Hardware Setup

7. Build the SA555P astable circuit: 5 V supply, RA=4.7 kΩ, RB=4.7 kΩ, C=100 nF
8. Add voltage divider on pin 3 output: 2 kΩ series, 3.3 kΩ to GND
9. Connect divider node to expansion connector pin 15 (package pin N14)
10. Connect USB-UART: board TX (C4) to PC
11. Power the board and 555 circuit

4.2 PulseView Verification

12. Probe pin 3 of SA555P with logic analyser
13. Set 2 MHz sample rate, 10M samples, run capture
14. Add Timing decoder — verify period, frequency, duty cycle
15. Note measured frequency as ground truth for FPGA comparison

4.3 FPGA Programming

16. Create RTL project in Vivado, target xc7a35tcsg324-1
17. Add uart_tx.v and top_555_freq.v
18. Add constraints.xdc
19. Synthesise → Implement → Generate Bitstream → Program

4.4 Expected UART Output (raw hex)

Each line is one period measurement. Example at ~1 kHz:

```
0000C350   ->  50,000 cycles  ->  1000.00 Hz
0000C34F   ->  49,999 cycles  ->  1000.02 Hz
```

5. Python PC-Side Decoder

A minimal Python script reads the UART hex stream from the FPGA and prints the live frequency in Hz. No headers, no timestamps — just the frequency on each line, updated every 555 cycle.

5.1 Requirements

- Python 3.7 or newer
- pyserial: pip install pyserial
- FPGA programmed and USB-UART connected
- No PuTTY / TeraTerm holding the COM port open

5.2 Script

Save as freq_reader.py and update COM_PORT to match your system:

```
import serial

COM_PORT   = 'COM11'      # Change to your actual COM port
BAUD_RATE  = 115200
FPGA_CLOCK = 50_000_000   # 50 MHz

def main():
    try:
        ser = serial.Serial(COM_PORT, BAUD_RATE, timeout=1)
        while True:
            if ser.in_waiting > 0:
                raw_data = ser.readline().decode('utf-8', errors='ignore').strip()
                if len(raw_data) == 8:
                    try:
                        clock_cycles = int(raw_data, 16)
                        if clock_cycles > 0:
                            frequency = FPGA_CLOCK / clock_cycles
                            print(f"{frequency:.2f} Hz")
                    except ValueError:
                        pass  # Silently ignore malformed data
    except serial.SerialException as e:
        print(f'Error: {e}')
    except KeyboardInterrupt:
        pass  # Silently exit on Ctrl+C
    finally:
        if 'ser' in locals() and ser.is_open:
            ser.close()

if __name__ == '__main__':
    main()
```

5.3 How It Works

Step	What happens
Open port	serial.Serial() opens COM port at 115200 baud
Read line	readline() reads until \n, strip() removes \r\n
Validate	len == 8 filters partial or corrupted lines
Convert	int(raw, 16) gives clock_cycles as integer
Compute & print	f = 50_000_000 / clock_cycles, printed as XX.XX Hz

5.4 Expected Output

Run with: python freq_reader.py

```
2599.03 Hz
2600.52 Hz
2598.76 Hz
2601.10 Hz
```

The small variation between readings is normal — it reflects the natural jitter of the SA555P (1.5–2.25% per datasheet). The FPGA measures with 20 ns resolution (one 50 MHz tick).

5.5 Troubleshooting

Problem	Fix
Error on open	Wrong COM port, or PuTTY/TeraTerm still open. Close all terminals first.
No output	FPGA not programmed, 555 not powered, or reset button held.
Garbled values	Baud rate mismatch. Check CLKS_PER_BIT in Verilog.
Frequency off by 2x	Check FPGA_CLOCK in Python — 50 MHz not 100 MHz.